Lab 2: Lexical Analyzer for Token Recognition Using Flex

Experiment Title

Lexical Analyzer for Token Recognition using Flex

Aim

To design and implement a lexical analyzer using Flex that reads a C-like program and identifies tokens such as:

- Keywords
- Identifiers
- Operators
- Numbers
- Special Symbols
- Separators

Software Requirements

Tool	Purpose
Flex	Lexical Analyzer Generator
GCC	C Compiler to compile generated code
OS	Windows (MinGW)

Install WinFlexBison

1. Go to: https://github.com/lexxmark/winflexbison/releases Download the latest win_flex_bison-X.X.X.zip file (e.g., win_flex_bison-2.5.25.zip)

2. Right-click the downloaded ZIP → Extract All. Extract to a permanent location, for example: C:\winflexbison\

3. Inside C:\winflexbison\ you should see: win_flex.exe, win_bison.exe, FlexLexer.h, data\

4. Add to System PATH
5. Verify in command prompt with win_flex --version

Theory

A lexical analyzer reads the source code character by character and groups characters into meaningful sequences called tokens.

Tokens in C include:

- Keywords: if, else, int, float, return, ...
- Identifiers: variable or function names
- Operators: +, -, *, /, =
- Numbers: 1, 2, 3, etc.
- Special Symbols: ;, (,), {, }, etc.

Flex

What is flex — a tool that takes your `.l` file (regex rules) and generates a complete C scanner. You describe tokens; flex builds the state machine.

File structure — every flex file has three sections split by `%%`: definitions → rules → user code. The rules section is where the real work happens.

Each section — definitions give you named regex aliases and C includes; rules pair patterns with actions; user code is just a `main()` calling `yylex()`.

Program Structure

Flex File (tokenizer.l)

```
%{
#include <stdio.h>
#include <string.h>

int line_no = 1;
%}

DIGIT      [0-9]
ID         [a-zA-Z_][a-zA-Z0-9_]*
KEYWORD    int|float|if|else|return
OP         ==|!=|<=|>=|\+|\-|\*|\/|>|=
SEPARATOR  [;,(){}]
```

```
%%

{KEYWORD}   { printf("Keyword: %s\n", yytext); }
{ID}        { printf("Identifier: %s\n", yytext); }
{DIGIT}+    { printf("Number: %s\n", yytext); }
{OP}        { printf("Operator: %s\n", yytext); }
{SEPARATOR} { printf("Separator: %s\n", yytext); }

\n          { line_no++; }

[ \t]       ;

.           { printf("Unknown symbol: %s\n", yytext); }

%%

int main() {
    printf("Enter the code (Ctrl+Z or Ctrl D to end input):\n");
    yylex();
    return 0;
}

int yywrap() {
    return 1;
}
```

Compilation and Execution Steps

```
win_flex tokenizer.l
gcc lex.yy.c -o tokenizer
./tokenizer
```

Sample Input / Output

Input:

```
int a = 5;
float b = a + 10;
if (b > 10) {
        return b;
}
```

Output:

```
Keyword: int        Identifier: a       Operator: =
```

```
Number: 5           Separator: ;        Keyword: float
Identifier: b       Operator: =         Identifier: a
Operator: +         Number: 10          Separator: ;
Keyword: if         Separator: (        Identifier: b
Operator: >         Number: 10          Separator: )
Separator: {        Keyword: return     Identifier: b
Separator: ;        Separator: }
```